MLOps – Assignment Report

Heart Disease Prediction API

Name: Shanmathy
ID: 2025cs05019
GitHub Repo: https://github.com/ShanmathySanjay/mlops-heart-disease
Demo Recording Link:
https://drive.google.com/file/d/1wqkRPKkklxRqb8Sl096AhdB8Pzri8gwO/view?usp=drive_link

---

1. Project Overview

This project implements an end-to-end MLOps pipeline for predicting heart disease using machine learning.

The system integrates:

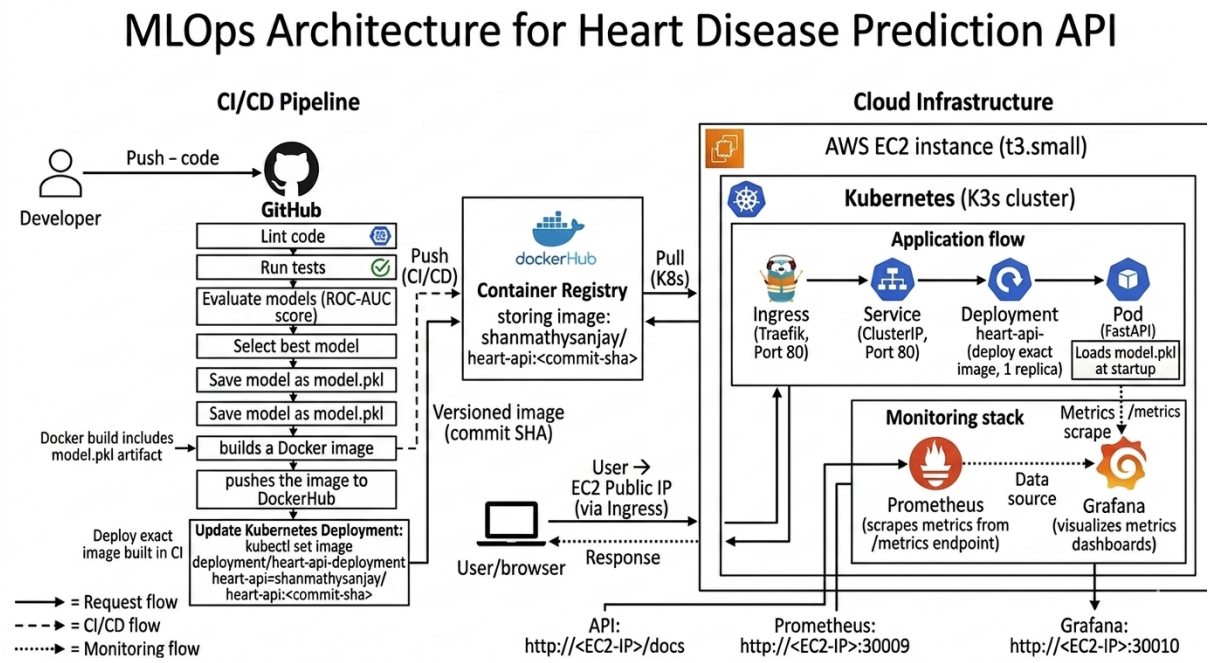
- Data preprocessing pipeline
- Model training and evaluation
- Experiment tracking using MLflow
- FastAPI-based inference API
- CI/CD automation using GitHub Actions
- Docker containerization
- Kubernetes deployment (K3s on AWS EC2)
- Monitoring using Prometheus and Grafana

The objective is to build a reproducible, scalable, and production-ready ML system.

2. Tech Stack

Category	Technology
Language	Python
API	FastAPI
ML	Scikit-learn
Tracking	MLflow (local + CI file store)
CI/CD	GitHub Actions
Containerization	Docker
Registry	DockerHub
Orchestration	Kubernetes (K3s)
Monitoring	Prometheus
Visualization	Grafana
Cloud	AWS EC2
Ingress	Traefik

3. Architecture



The system follows a modern cloud-native MLOps architecture designed for scalability, reproducibility, and continuous delivery. The workflow begins with a developer pushing code changes to the GitHub repository, which automatically triggers the CI/CD pipeline using GitHub Actions. The pipeline performs code linting, testing, and model training, after which a Docker image is built containing the trained model artifact (model.pkl). This image is versioned using the commit SHA and pushed to DockerHub, ensuring traceability and reproducibility.

On the deployment side, the system is hosted on an AWS EC2 instance running a lightweight Kubernetes (K3s) cluster. The Kubernetes cluster pulls the latest Docker image from DockerHub and updates the running application using a rolling deployment strategy. Within the cluster, the application is exposed through a Kubernetes Service, while Traefik Ingress manages external access by routing incoming HTTP requests to the appropriate service.

At runtime, user requests are sent to the public EC2 endpoint, which passes through the Ingress layer to the Service and then to the FastAPI-based application pod. The application loads the trained model at startup and performs real-time predictions based on incoming input data.

For observability, Prometheus continuously scrapes metrics from the application's /metrics endpoint, such as request counts and request rates. These metrics are visualized in Grafana dashboards, enabling real-time monitoring of system performance and usage patterns.

Overall, this architecture ensures a seamless flow from code integration to deployment, while maintaining reproducibility, scalability, and monitoring capabilities essential for production-grade machine learning systems.

4. Setup / Installation

4.1 Clone Repository

```
git clone https://github.com/ShanmathySanjay/mlops-heart-disease.git
cd mlops-heart-disease
```

4.2 Install Dependencies

```
pip install -r requirements.txt
```

4.3 Train Model

```
PYTHONPATH=. python src/models/train.py
```

This performs:

- Data loading
- Preprocessing
- Model training
- Cross-validation
- Model selection
- MLflow logging

4.4 MLflow UI (Experiment Tracking)

```
mlflow ui
```

Open in browser:

http://127.0.0.1:5000

Shows:

- Metrics (accuracy, ROC-AUC, etc.)
- Parameters
- Artifacts (confusion matrix, ROC curves)

4.5 Run API

```
uvicorn src.api.app:app --reload
```

Swagger UI:

http://localhost:8000/docs

4.6 Run Tests

pytest tests/

4.7 Docker

docker build -t heart-api .
docker run -p 8000:8000 heart-api

4.8 Kubernetes Deployment

kubectl apply -f k8s-deployment.yaml
kubectl apply -f ingress.yaml
kubectl apply -f prometheus.yaml
kubectl apply -f grafana.yaml

4.9 Verify Deployment

kubectl get pods
kubectl get svc
kubectl get ingress

4.10 Prometheus (Monitoring Backend)

Access via NodePort:

http://<EC2-IP>:30009

Query: request_count_total

4.11 Grafana (Visualization Dashboard)

Access:

http://<EC2-IP>:30010

Setup:

- Add Prometheus as data source
- Create dashboard
- Query: rate(request_count_total[1m])

5. Dataset Overview

The dataset used for this project is the Heart Disease dataset, which contains clinical and demographic information used to predict the presence of heart disease.

- Dataset: UCI Heart Disease Dataset
- Source: https://archive.ics.uci.edu/ml/datasets/heart+Disease
- Subset Used: processed.cleveland.data

- **Number of samples:** 303
- **Number of features:** 14 (including target)
- **Target variable:** target
    - 0 → No heart disease
    - 1 → Presence of heart disease

The dataset contains 14 attributes including age, sex, chest pain type, cholesterol levels, and other medical indicators. The target variable indicates the presence of heart disease.

5.1 Initial Observations

- Dataset is relatively small and structured
- Contains both categorical and continuous variables
- Some missing values are present
- Target is moderately balanced

The dataset is included in the repository under the /data directory for reproducibility.

6. Data Preprocessing

The following preprocessing steps were applied:
- Assigned meaningful column names to the dataset
- Handled missing values (`ca` and `thal`) by removing rows with missing entries

- Converted the target variable into a binary classification:

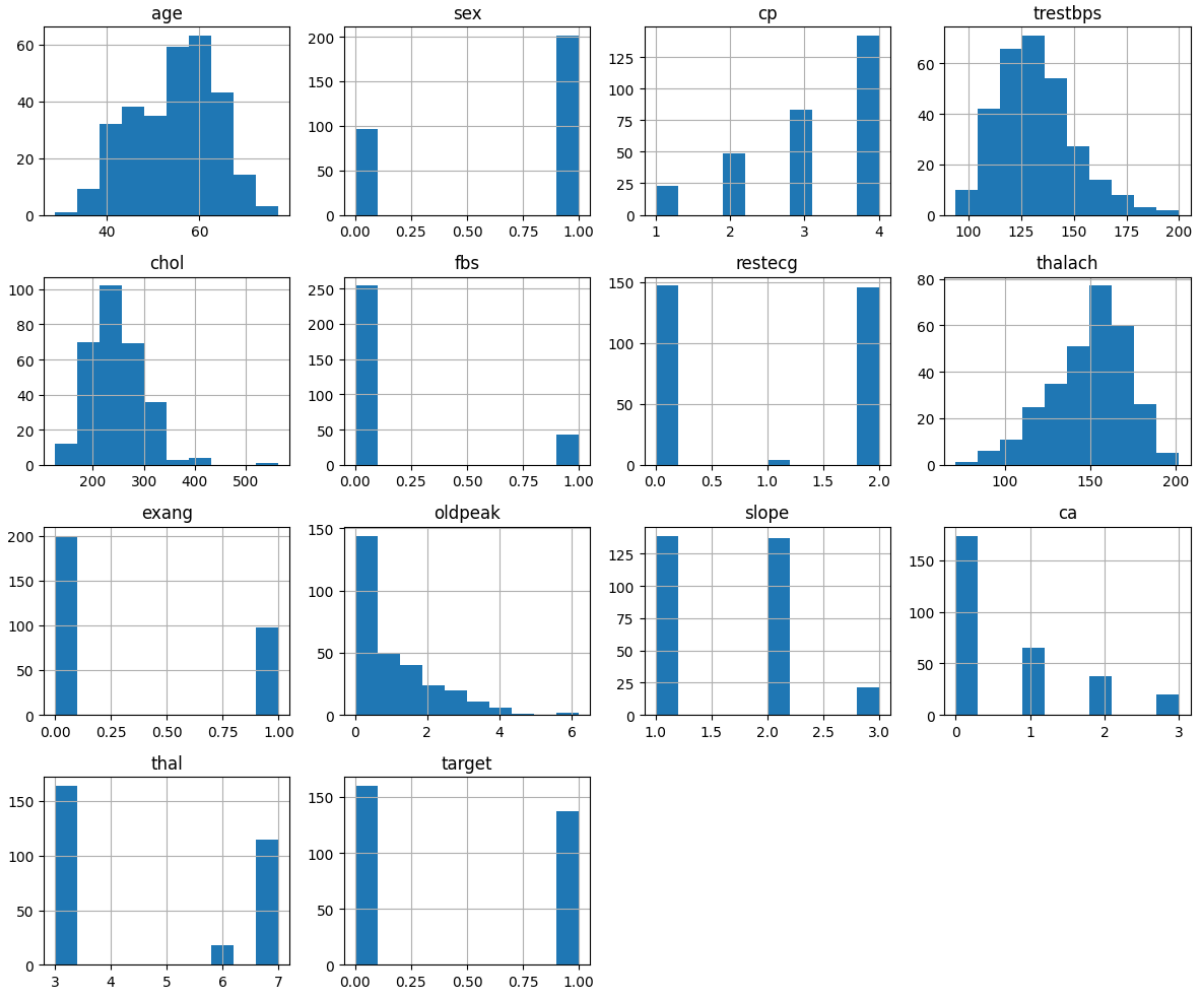
- 0 → No heart disease
- 1 → Presence of heart disease
- Split data into features (X) and target (y)

All features are numerical; hence no categorical encoding was required.

7. Exploratory Data Analysis (EDA)

EDA Steps Performed

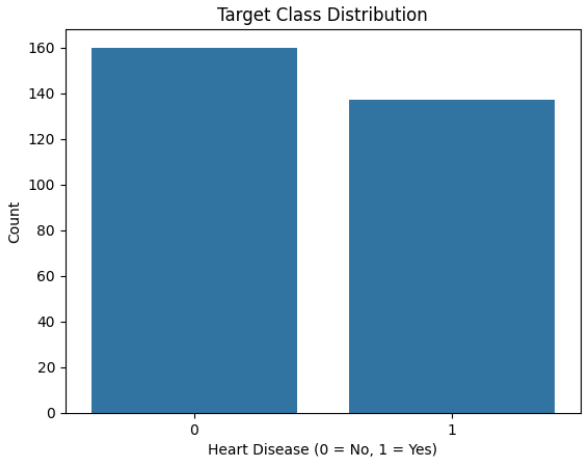
7.1 Feature Distributions



Observations:

- Age and cholesterol show variation
- Some features are skewed
- Mix of categorical and numerical features

7.2 Target Distribution

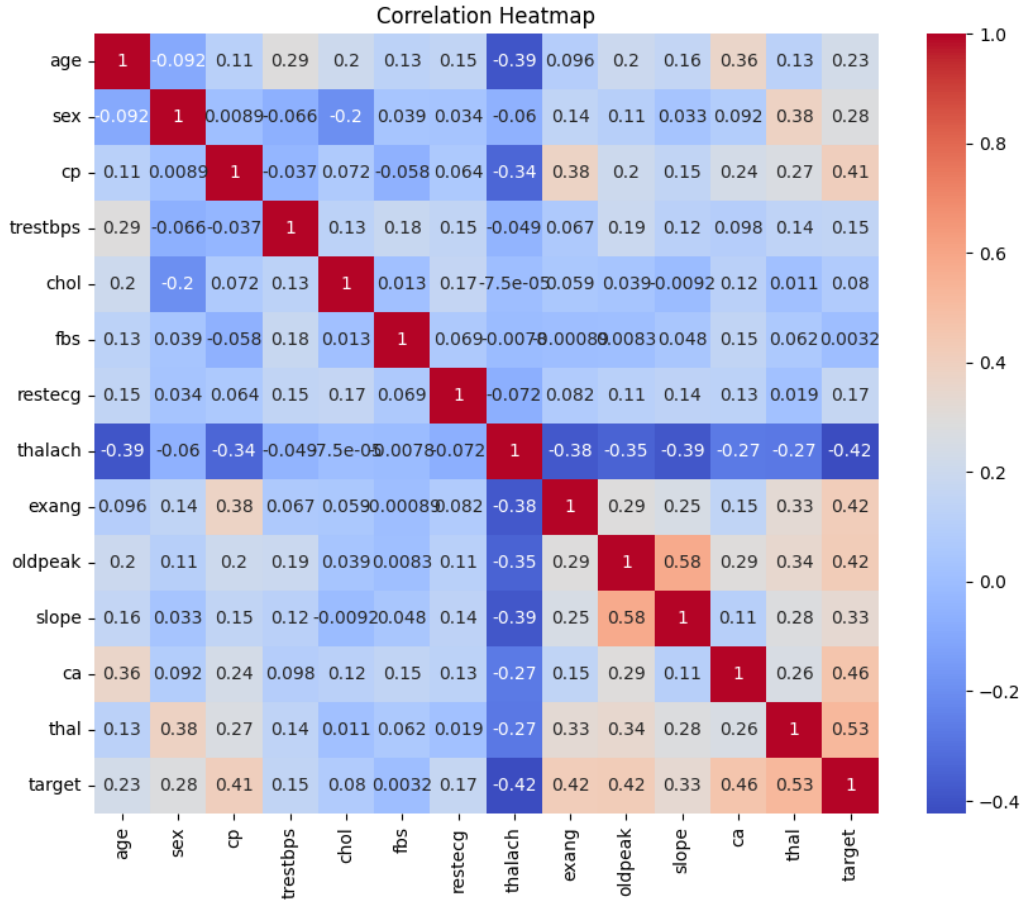


Class distribution:

- 160 (No disease)
- 137 (Disease)

- Dataset is **reasonably balanced**

7.3 Correlation Analysis



Key insights:

- Strong correlations:
  - **thal**
  - **ca**
  - **oldpeak**
  - **thalach (negative correlation)**
- These features are important predictors

8. Modelling Approach

The modelling phase focuses on developing, evaluating, and selecting the most suitable machine learning model for heart disease prediction. Two supervised classification algorithms were implemented using Scikit-learn: Logistic Regression and Random Forest.

To ensure consistency and reproducibility, both models were integrated into a structured pipeline.

8.1 Pipeline Design

A Scikit-learn Pipeline was used to standardize the workflow:

StandardScaler → Model

The inclusion of a pipeline ensures that preprocessing steps such as feature scaling are consistently applied during both training and inference.

The use of pipelines provides several advantages:

- Prevents data leakage by ensuring transformations are applied only on training data
- Improves reproducibility across environments
- Simplifies model deployment

8.2 Models Implemented

Two models were selected to compare performance across different learning paradigms:

- **Logistic Regression**
  A linear model that serves as a baseline. It is simple, interpretable, and performs well when relationships between features and target are approximately linear.
- **Random Forest Classifier**
  An ensemble learning method that builds multiple decision trees and aggregates their outputs. It captures non-linear relationships and is robust to noise and feature interactions.

8.3 Training Strategy

The dataset was split into training and testing sets using an 80:20 ratio.
Logistic Regression was trained with feature scaling, while Random Forest was trained without scaling, as tree-based models are invariant to feature magnitude.

9. Evaluation and Model Selection

9.1 Evaluation Metrics

To comprehensively evaluate model performance, multiple metrics were used:

- **Accuracy** – Overall correctness of predictions
- **Precision** – Proportion of correct positive predictions
- **Recall** – Ability to correctly identify positive cases
- **ROC-AUC Score** – Measures classification performance across all thresholds

These metrics provide a balanced view of model performance, especially for healthcare applications where false negatives are critical.

9.2 Cross-Validation

To ensure robustness and generalization, 5-fold cross-validation was performed:

cross_val_score(model, X, y, cv=5)

The following were computed and logged:

- Mean cross-validation score
- Standard deviation

Cross-validation reduces dependency on a single train-test split and provides a more reliable estimate of model performance.

9.3 Results

Model	Accuracy	Precision	Recall	ROC-AUC
Logistic Regression	0.866	0.833	0.833	0.861
Random Forest	0.883	0.840	0.875	0.882

9.4 Model Selection

The final model was selected based on ROC-AUC score, as it reflects overall classification performance.

The Random Forest model achieved the highest ROC-AUC and better recall, indicating improved capability in identifying positive heart disease cases.

The selected model was saved as:
model.pkl

10. Experiment Tracking (MLflow)

MLflow was used to track experiments, ensuring reproducibility and transparency throughout the model development lifecycle.

10.1 Logging Strategy

Each model training run logs:

- **Parameters**
  - Model type
  - Hyperparameters (e.g., max_iter, n_estimators)
- **Metrics**
  - Accuracy
  - Precision
  - Recall
  - ROC-AUC
  - Cross-validation mean and standard deviation
- **Artifacts**
  - Confusion matrix plots
  - ROC curve visualizations
  - Trained model files

10.2 Experiment Structure

Separate runs were maintained for:

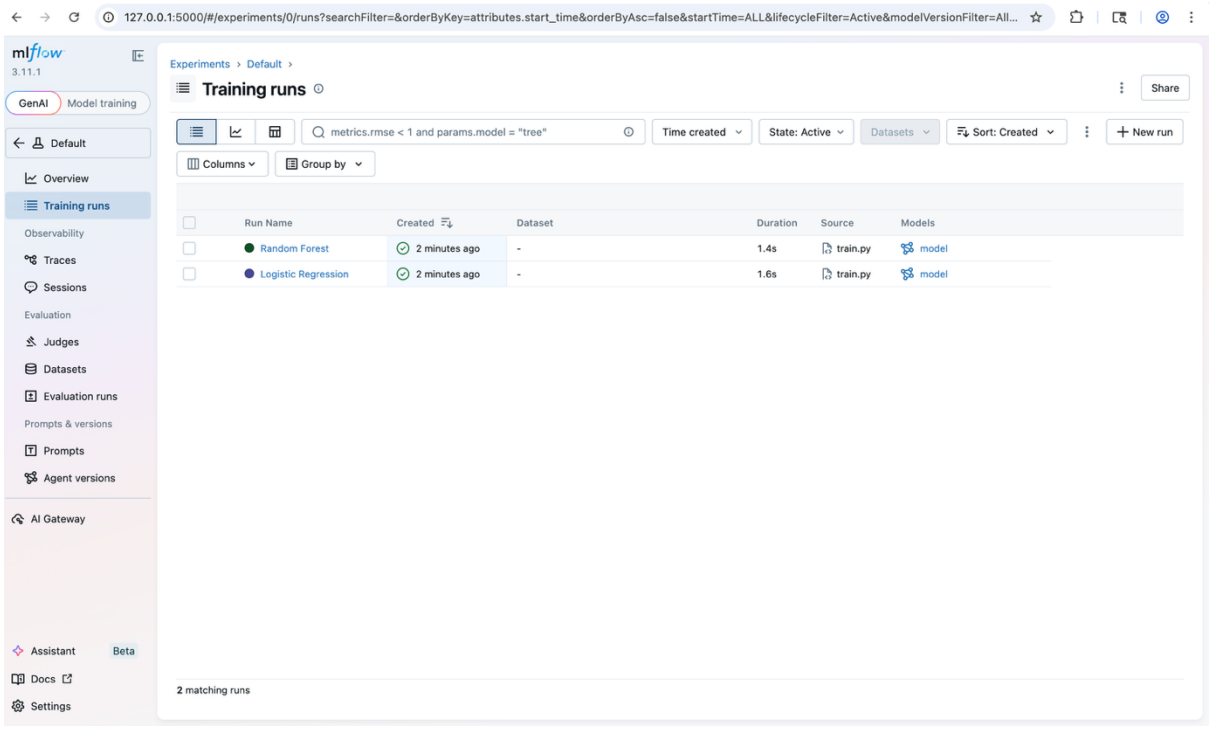
- Logistic Regression
- Random Forest

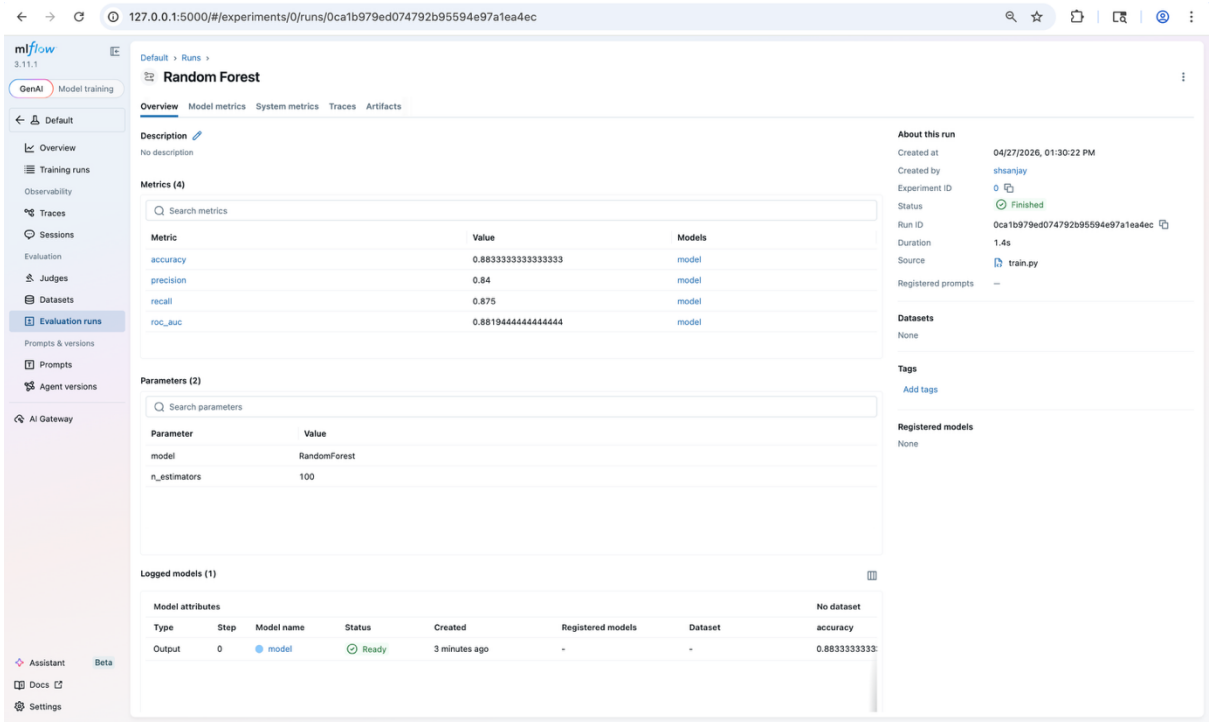
A final run logs the best model, enabling clear traceability.

10.3 Tracking Modes

- Local tracking UI:
  http://127.0.0.1:5000
- CI/CD tracking:
  Stored in local file system (mlruns directory)

MLflow enables comparison between multiple runs, helping identify the best-performing model efficiently.


127.0.0.1:5000/#/experiments/0/runs?searchFilter=&orderByKey=attributes.start_time&orderByAsc=false&startTime=ALL&lifecycleFilter=Active&modelVersionFilter=All...
mlflow
3.11.1
GenAI
Model training
Default
Overview
Training runs
Observability
Traces
Sessions
Evaluation
Judges
Datasets
Evaluation runs
Prompts & versions
Prompts
Agent versions
AI Gateway
Assistant
Beta
Docs
Settings
Experiments › Default ›
Training runs
Share
metrics.rmse < 1 and params.model = "tree"
Time created
State: Active
Datasets
Sort: Created
New run
Columns
Group by
Run Name
Created
Dataset
Duration
Source
Models
Random Forest
2 minutes ago
-
1.4s
train.py
model
Logistic Regression
2 minutes ago
-
1.6s
train.py
model
2 matching runs


127.0.0.1:5000/#/experiments/0/runs/0ca1b979ed074792b95594e97a1ea4ec
Default › Runs ›
Random Forest
Overview
Model metrics
System metrics
Traces
Artifacts
Description
No description
Metrics (4)
Search metrics
Metric
Value
Models
accuracy
0.8833333333333333
model
precision
0.84
model
recall
0.875
model
roc_auc
0.8819444444444444
model
Parameters (2)
Search parameters
Parameter
Value
model
RandomForest
n_estimators
100
Logged models (1)
Model attributes
No dataset
Type
Step
Model name
Status
Created
Registered models
Dataset
accuracy
Output
0
model
Ready
3 minutes ago
-
-
0.8833333333
About this run
Created at
04/27/2026, 01:30:22 PM
Created by
shsanjay
Experiment ID
0
Status
Finished
Run ID
0ca1b979ed074792b95594e97a1ea4ec
Duration
1.4s
Source
train.py
Registered prompts
–
Datasets
None
Tags
Add tags
Registered models
None

127.0.0.1:5000/#/experiments/0/runs/87e5a3ef7db5452f8b7654bc1a27eed8

mlflow 3.11.1

GenAI | Model training

Default

Overview
Training runs
Observability
Traces
Sessions
Evaluation
Judges
Datasets
Evaluation runs
Prompts & versions
Prompts
Agent versions
AI Gateway

Assistant Beta
Docs
Settings

Default › Runs ›

Logistic Regression

Overview | Model metrics | System metrics | Traces | Artifacts

Description

No description

Metrics (4)

Search metrics

Metric	Value	Models
accuracy	0.8666666666666667	model
precision	0.8333333333333334	model
recall	0.8333333333333334	model
roc_auc	0.8611111111111112	model

Parameters (2)

Search parameters

Parameter	Value
model	LogisticRegression
max_iter	1000

Logged models (1)

Model attributes							No dataset
Type	Step	Model name	Status	Created	Registered models	Dataset	accuracy
Output	0	model	Ready	4 minutes ago	-	-	0.866666666

About this run

Created at: 04/27/2026, 01:30:21 PM
Created by: shsanjay
Experiment ID: 0
Status: Finished
Run ID: 87e5a3ef7db5452f8b7654bc1a27eed8
Duration: 1.6s
Source: train.py
Registered prompts: –

Datasets

None

Tags

Add tags

Registered models

None

127.0.0.1:5000/#/experiments/0/runs/67d0d4f5dfb14fbe878e46cbd0b55dc4/artifacts

mlflow 3.11.1

GenAI | Model training

Default

Overview
Training runs
Observability
Traces
Sessions
Evaluation
Judges
Datasets
Evaluation runs
Prompts & versions
Prompts
Agent versions
AI Gateway

Assistant Beta
Docs
Settings

Default › Runs ›

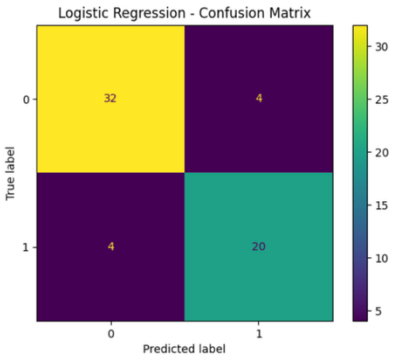
Logistic Regression

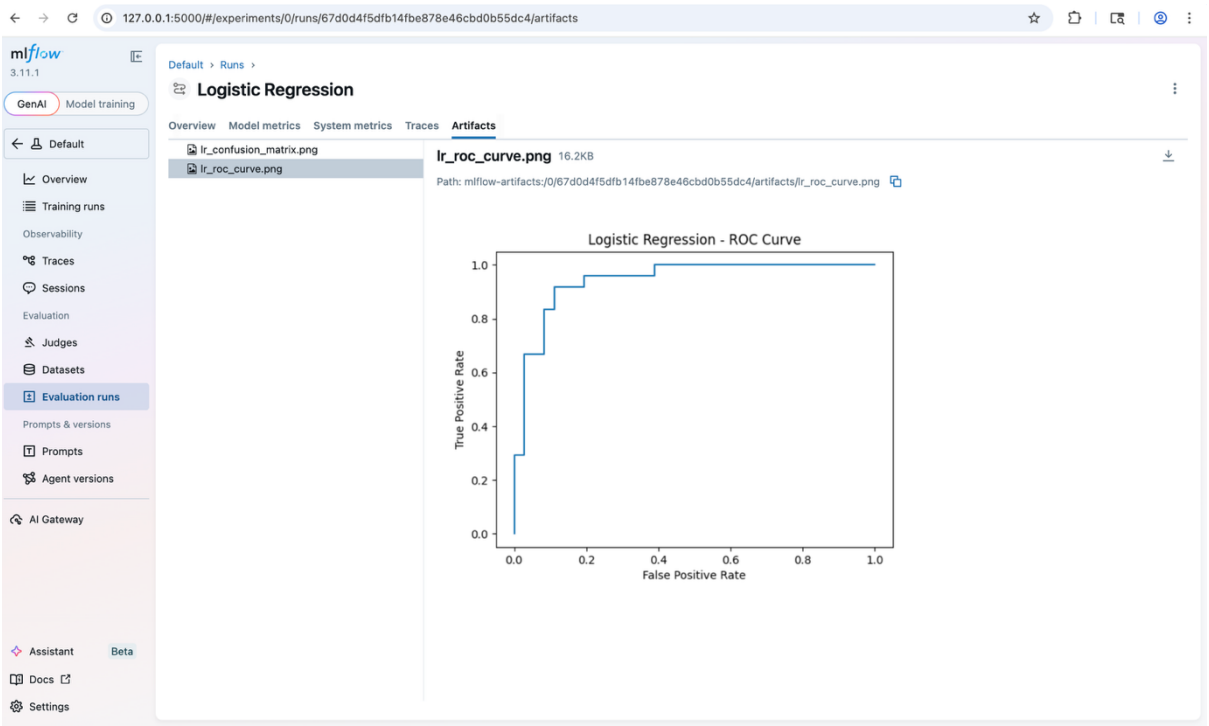
Overview | Model metrics | System metrics | Traces | Artifacts

lr_confusion_matrix.png
lr_roc_curve.png

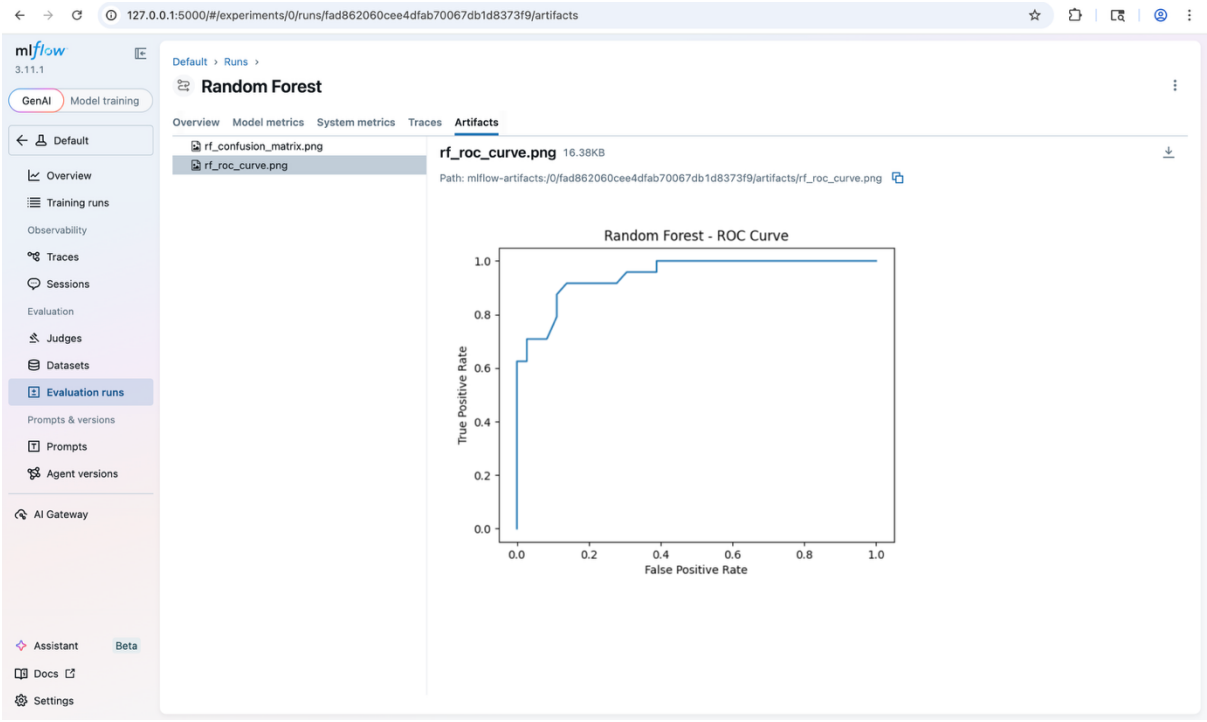
lr_confusion_matrix.png 16.08KB

Path: mlflow-artifacts:/0/67d0d4f5dfb14fbe878e46cbd0b55dc4/artifacts/lr_confusion_matrix.png




127.0.0.1:5000/#/experiments/0/runs/67d0d4f5dfb14fbe878e46cbd0b55dc4/artifacts
mlflow
3.11.1
GenAI
Model training
Default
Overview
Training runs
Observability
Traces
Sessions
Evaluation
Judges
Datasets
Evaluation runs
Prompts & versions
Prompts
Agent versions
AI Gateway
Assistant
Beta
Docs
Settings
Default › Runs ›
Logistic Regression
Overview
Model metrics
System metrics
Traces
Artifacts
lr_confusion_matrix.png
lr_roc_curve.png
lr_roc_curve.png 16.2KB
Path: mlflow-artifacts:/0/67d0d4f5dfb14fbe878e46cbd0b55dc4/artifacts/lr_roc_curve.png
Logistic Regression - ROC Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0


127.0.0.1:5000/#/experiments/0/runs/fad862060cee4dfab70067db1d8373f9/artifacts
mlflow
3.11.1
GenAI
Model training
Default
Overview
Training runs
Observability
Traces
Sessions
Evaluation
Judges
Datasets
Evaluation runs
Prompts & versions
Prompts
Agent versions
AI Gateway
Assistant
Beta
Docs
Settings
Default › Runs ›
Random Forest
Overview
Model metrics
System metrics
Traces
Artifacts
rf_confusion_matrix.png
rf_roc_curve.png
rf_confusion_matrix.png 15.26KB
Path: mlflow-artifacts:/0/fad862060cee4dfab70067db1d8373f9/artifacts/rf_confusion_matrix.png
Random Forest - Confusion Matrix
True label
Predicted label
32
4
3
21
0
1
30
25
20
15
10
5



11. CI/CD Pipeline

The CI/CD pipeline automates the entire workflow from code integration to deployment.

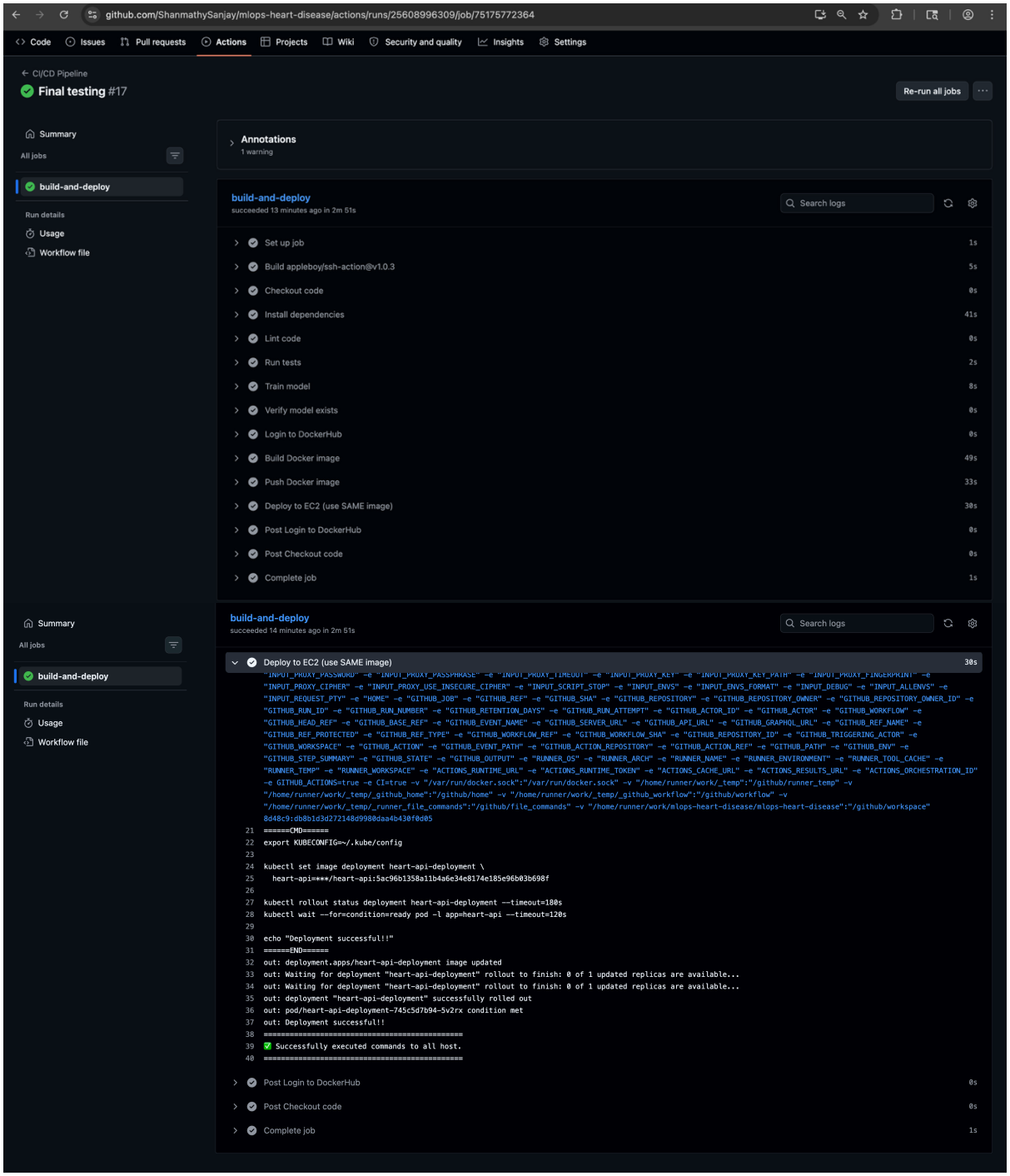
11.1 Workflow Overview

The pipeline is triggered on every code push and performs the following steps:

1. Install dependencies
2. Lint code using flake8
3. Execute unit tests
4. Train machine learning model
5. Save model artifact (model.pkl)
6. Build Docker image
7. Push image to DockerHub
8. Deploy updated image to Kubernetes

11.2 Automation Benefits

- Ensures continuous integration and validation
- Eliminates manual deployment steps
- Guarantees consistency between development and production



12. Deployment Architecture

The system is deployed on an AWS EC2 instance using Kubernetes (K3s).

12.1 Infrastructure Components

- **EC2 Instance (t3.small)** – Hosting environment
- **Kubernetes (K3s)** – Container orchestration
- **Docker** – Application containerization
- **Traefik Ingress** – Routing external traffic

12.2 Request Flow

User → Ingress → Service → Pod → FastAPI → Model → Response

- User requests are routed through Ingress
- Service forwards requests to application pods
- FastAPI loads the trained model and returns predictions

12.3 Kubernetes Resources

- Deployment → manages application pods
- Service → internal communication
- Ingress → external access

```
ubuntu@ip-172-31-1-23:~$
ubuntu@ip-172-31-1-23:~$ kubectl get pods
NAME                                  READY   STATUS    RESTARTS   AGE
grafana-894f94594-6zljn               1/1     Running   0          22m
heart-api-deployment-696d9878d5-dhzhq   1/1     Running   0          24m
prometheus-5dfb9956fd-jggsv           1/1     Running   0          22m
ubuntu@ip-172-31-1-23:~$ kubectl get svc
NAME                 TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
grafana-service      NodePort    10.43.123.153   <none>        3000:30010/TCP   22m
heart-api-service    ClusterIP   10.43.101.185   <none>        80/TCP           12h
kubernetes           ClusterIP   10.43.0.1       <none>        443/TCP          12h
prometheus-service   NodePort    10.43.46.36     <none>        9090:30009/TCP   22m
ubuntu@ip-172-31-1-23:~$ kubectl get ingress
NAME                CLASS     HOSTS   ADDRESS        PORTS   AGE
heart-api-ingress   traefik   *       172.31.1.23    80      12h
ubuntu@ip-172-31-1-23:~$
```

13. API Service

The system exposes a REST API using FastAPI.

13.1 Endpoint

POST /predict

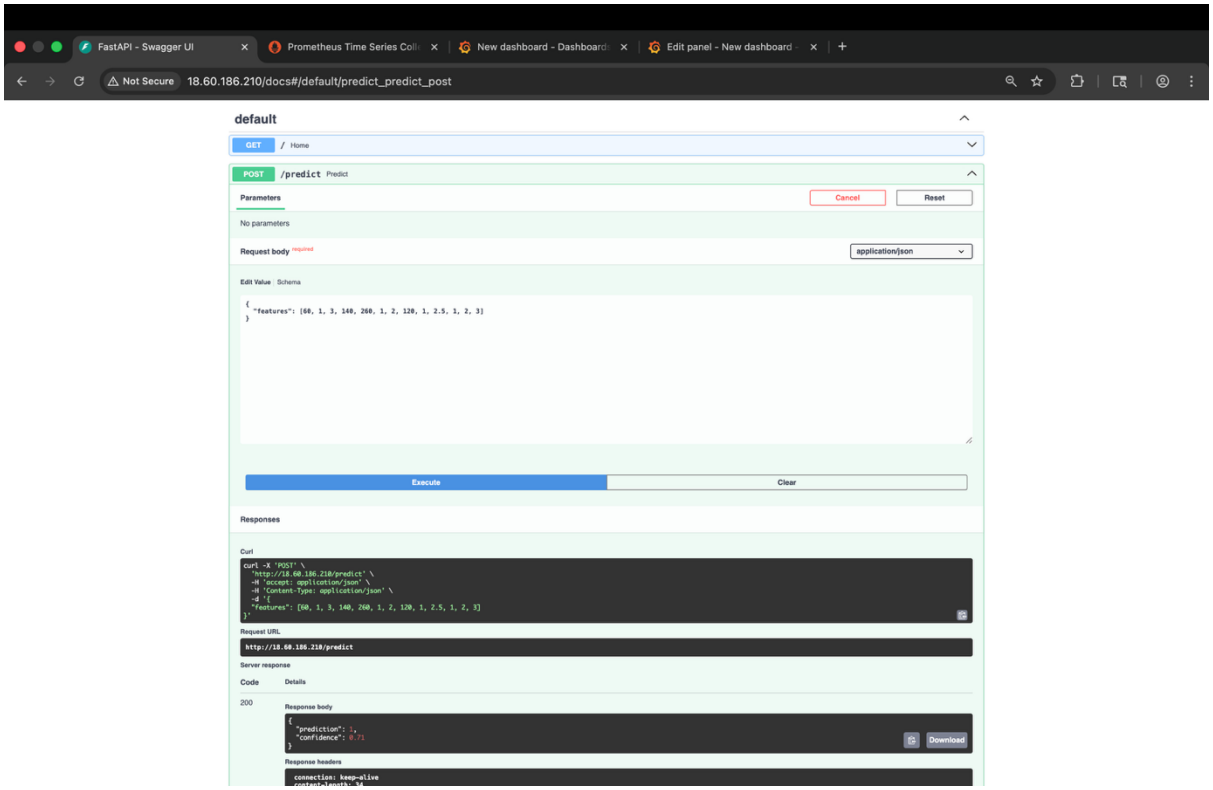
13.2 Input Format

```
{
 "features": [60, 1, 3, 140, 260, 1, 2, 120, 1, 2.5, 1, 2, 3]
}
```

13.3 Output Format

```
{
 "prediction": 1,
 "confidence": 0.71
}
```

The API performs real-time inference by loading the trained model during application startup.



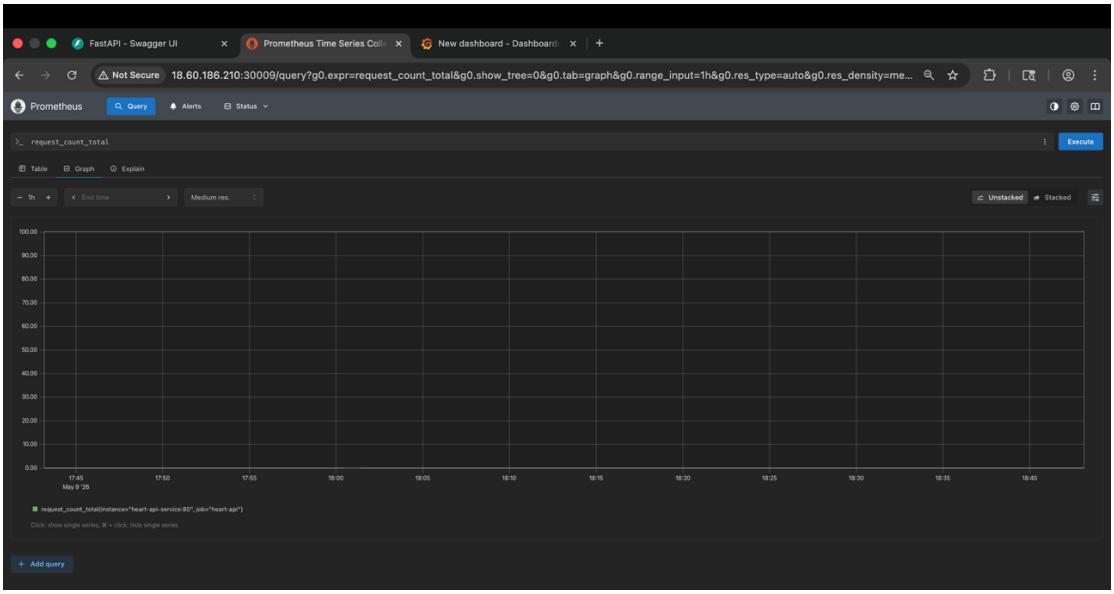
14. Monitoring and Observability

Monitoring is implemented using Prometheus and Grafana.

14.1 Prometheus

Prometheus scrapes metrics from the application's /metrics endpoint.

Tracked metric: request_count_total



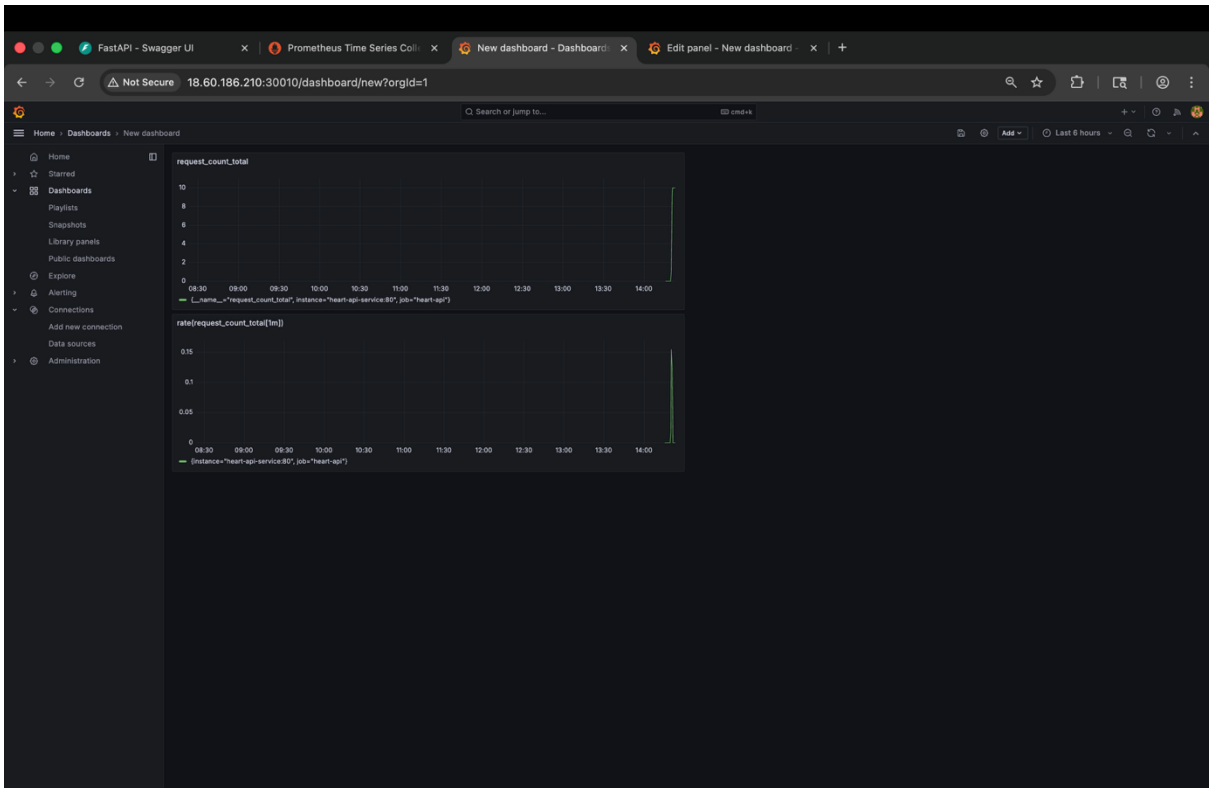
14.2 Grafana

Grafana provides visualization dashboards for monitoring system performance.

Key visualizations:

- Total request count
- Request rate:
  rate(request_count_total[1m])

This enables real-time tracking of API usage and system behavior.



15. Production Readiness

The system incorporates several features that make it production-ready:

- Reproducible training pipeline
- Automated CI/CD workflow
- Containerized deployment
- Kubernetes orchestration
- Experiment tracking with MLflow
- Real-time monitoring and observability
- Versioned model artifacts

16. Conclusion

This project demonstrates a complete end-to-end MLOps pipeline for heart disease prediction. It integrates data preprocessing, model training, evaluation, experiment tracking, automated deployment, and monitoring into a unified workflow.

The system ensures:

- Scalability through Kubernetes
- Reproducibility through pipelines and MLflow
- Reliability through CI/CD automation
- Observability through monitoring tools

Such an architecture closely resembles real-world production systems and provides a strong foundation for deploying machine learning solutions in practical environments.